



SENIOR SOFTWARE ENGINEER INTERVIEW QUESTIONS: DEEP DIVES

Architecture Challenge

The senior engineer answer sheet

Nine changes to one design. The first-order answer is easy. That is not what is being marked.

Each challenge below changes one thing about a working payment pipeline. Answer it before reading on.

The first-order answer is easy and almost everyone gets it. The interview is decided by the second-order effect, and by the two questions at the end that no blog post can prepare you for.

Kafka behaviour is sourced to Apache Kafka 4.3 documentation. Reasoning that is reasoning says so.

In scope

- Consumers against partitions, and the parallelism ceiling
- Changing the partition count, and its three consequences
- Crash after the effect
- Retry topics against business ordering
- The ambiguous external call
- The relay that stops
- A second consumer group
- The two bonus traps

Out of scope

- Rebuilding the pipeline itself, which is episode 9
- Cluster operations, broker tuning and capacity
- Kafka Streams, which changes several of these answers
- Provider specific payment API behaviour

Every challenge here has a correct short answer and a better long one. The long one names what it cost.

01 Worked assumptions

Starting partitions	3
Consumers deployed	6
Consumers doing work	3
Consumers idle	3
Partitions after the change	6
Partitions that can ever be removed	0

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.

03 One: three partitions, six consumers

Three consumers get one partition each. Three sit idle. A rebalance happened and achieved nothing.

Partition count is the parallelism CEILING for one consumer group. Consumers beyond it are standby capacity, not speed.

That is not waste, incidentally. Standby capacity fails over faster than a new pod starts. It is just not throughput.

Each partition is consumed by exactly one member of a consumer group, so the number of partitions bounds the number of usefully active consumers in that group.

Kafka 4.3, Design: Message Delivery Semantics

Adding consumers beyond the partition count triggers a rebalance without increasing throughput.

First principles, not vendor documentation

04 Two: three partitions become six, and this is the best question here

FIRST ORDER: the ceiling doubled.

SECOND ORDER, and this is the answer being marked: the same key may now hash to a different partition, so records for one account can end up split across two partitions with no ordering promise between them. Old records stay where they were. New ones may not.

THIRD: you can never reduce the count again. Whatever you pick, you are keeping.

Anyone can say more parallelism. The interview turns on what it cost.

The number of partitions for a topic can be increased but never decreased.

Kafka 4.3, Topics and Logs

Under the default partitioner a keyed record is assigned by hashing the key over the current partition count, so changing the count changes where existing keys land.

Kafka 4.3, Producer Configs

Ordering is guaranteed within a partition, so a key whose records span two partitions has no ordering guarantee between them.

Kafka 4.3, Topics and Logs



05 Three: the effect succeeds, then the consumer dies

It replays. That part is not in doubt and it is not a defect.

Whether the customer is charged twice is decided entirely by YOUR idempotency, not by anything in Kafka.

An answer that reaches for a Kafka setting here has misplaced the boundary, which is the thing the question is testing.

Kafka guarantees at-least-once delivery by default, so a record processed before an uncommitted crash is delivered again.

Kafka 4.3, Design: Message Delivery Semantics

Whether a redelivery produces a second effect is a property of the application, not of the broker.

First principles, not vendor documentation

06 Four: move failures to a retry topic

Event A fails and is delayed thirty seconds. Event B for the same account succeeds immediately. A returns and succeeds SECOND.

Partition ordering intact. Business ordering inverted.

That is a real trade rather than a bug, and the answer they want is you making it knowingly and naming the accounts or operations it is unacceptable for.

Retry topics preserve partition ordering while reordering the work, because the delayed record rejoins after later records have been processed.

First principles, not vendor documentation

Retrying in place instead preserves work ordering and blocks the partition behind the failing record.

First principles, not vendor documentation



07 Five: the external provider response is lost

The request left. The provider may have charged the customer. You know nothing.

Retry only if they will deduplicate, which means passing YOUR key to them and requiring it to be honoured.

This is the ambiguous failure, the hardest class in distributed systems, and it is only safe because the key exists. Without it the honest answer is that you cannot retry and cannot proceed, and a human has to reconcile.

A lost response leaves the caller unable to distinguish a request that was not processed from one that was processed with an undelivered acknowledgement.

First principles, not vendor documentation

Passing a caller-generated idempotency key to a remote participant is what converts an ambiguous failure into a safely retryable one.

First principles, not vendor documentation

08 Six: the outbox relay stops

Payments keep committing. Kafka is healthy. Consumers are healthy and idle.

Nothing is lost, and everything stops arriving.

Only the age of the oldest unpublished row can see it. If you have not built that panel, this outage is discovered by a customer.

A stopped relay between a database and Kafka produces no error in either system, because neither is failing.

First principles, not vendor documentation

The age of the oldest unpublished outbox row is the only local signal that detects a stopped relay.

First principles, not vendor documentation

09 Seven: add a second consumer group

No, analytics does not steal messages from notifications. Consumer groups are independent, each with its own committed offsets over the same records.

This one is genuinely simple, and it is here on purpose.

Not every question has a trap, and a candidate who invents one to look thorough has just demonstrated the opposite.

Each consumer group maintains its own committed offsets, so multiple groups consume the same records independently.

Kafka 4.3, Design: Message Delivery Semantics

A record is retained according to the topic retention policy rather than being removed on consumption, which is what makes independent groups possible.

Kafka 4.3, Topics and Logs

10 The two bonus traps

"WE ENABLED TRANSACTIONS, SO IT IS EXACTLY ONCE." Your first question is not how. It is: exactly once ACROSS WHAT? Kafka transactions cover records and input offsets inside Kafka. They do not reach a database or an HTTP call.

"CAN WE REPLAY FROM THE DEAD LETTER TOPIC?" Only if the identity outlived the record. Is the claim still present, or has it been cleaned up? Does the replay preserve the original key? Is anything downstream order sensitive about it arriving now?

That second one is the subtlest question in the series, and it is the one most likely to be waved through in a real review.

A Kafka transaction can atomically produce records to multiple topic partitions and commit the consumed input offsets in the same transaction.

Kafka 4.3, Design: Transactions

Exactly-once delivery for other destination systems generally requires cooperation with such systems, but Kafka provides the primitives which makes implementing this feasible.

Kafka 4.3, Design: Message Delivery Semantics

A replay is safe only where the idempotency identity for that operation still exists; if claims have been cleaned up, the replay is a new first attempt.

First principles, not vendor documentation

11 The final trap, and it is the one worth keeping

Do not copy every mechanism in this series into every Kafka application.

A low risk analytics pipeline does not need a payments architecture. Idempotency claims, an outbox, retry topics and invariant checks all cost something to build and more to operate, and on the wrong workload that cost buys nothing.

Knowing which of these to LEAVE OUT is the same skill as knowing how to build them, and it is the question with the highest signal in this entire sheet: what would you not build here, and why?

Every reliability mechanism carries a build and operational cost, so applying one where its failure mode is not material is a net loss.

First principles, not vendor documentation

Articulating what a design deliberately omits, and why, distinguishes an engineer who has operated systems from one who has read about them.

Operational practice, not vendor documentation

The seven questions to ask of any design

1. What is the ordering boundary, and what chose it?
2. What identifies one logical operation?
3. When does progress become durable, relative to the effect?
4. Which failures retry, and which never do?
5. Where does atomicity stop?
6. What is invisible while everything is green?
7. What would you deliberately not build here?
8. And for each answer above: what did it cost?



Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



X

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Apache Kafka and the Kafka logo are trademarks of the Apache Software Foundation. PostgreSQL and its logo are trademarks of the PostgreSQL Community Association. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.